HostelFlow — System Workflow Guide

Complete Operational Blueprint, Architecture & User Journey

1. Project Overview & Core Vision

HostelFlow is an end-to-end, paperless Online Hostel Management System (HMS) developed to eliminate physical paperwork, manual office queues, and untracked room allocations. It creates a digital, auditable bridge between college students and hostel wardens.

2. Dual-Role Architecture

- **Student Portal:** Students register, browse live non-AC rooms across 4 sharing tiers, apply online, receive auto-generated fee challans, pay via the MMMUT Samarth Fee Portal redirect, upload transaction receipts, and track room maintenance tickets.
- **Warden Control Room:** Wardens monitor total hostel occupancy, approve/reject student room applications, audit submitted transaction proofs via an in-app viewer, and manage maintenance complaints on a 3-stage Kanban board.

3. Step-by-Step End-to-End Workflow

- **Step 1: Student Registration & Login:** Students register with their college email, roll number, course, and year. Authentication uses secure bcrypt password hashing and JWT bearer tokens.
- **Step 2: Room Inventory Browsing:** Students filter rooms by sharing category (Single, Double, Triple, 4-Sharing). Live capacity counters prevent overbooking.
- **Step 3: Room Application:** Student chooses their preferred tier and clicks Apply. The system strictly enforces a single active application rule.
- **Step 4: Warden Atomic Approval:** Warden reviews pending requests in the Applications Queue. On approval, the system executes an atomic database transaction: verifies vacancy (occupied < capacity), locks the seat, assigns the room, and auto-generates a unique payment challan (e.g. HMS2026-1042-xxxx).
- **Step 5: External Smart Portal Payment:** Student receives the challan, clicks "Open MMMUT Samarth Fee Portal", completes the institutional payment externally, and submits their transaction ID and proof screenshot.
- **Step 6: Warden Audit & Confirmation:** Warden opens the Payment Verification Queue, clicks "View Proof" to inspect the receipt in an in-app modal, and confirms payment. The room status is permanently locked as Verified.
- **Step 7: Maintenance Desk:** Students can report electrical, plumbing, or Wi-Fi issues. The warden advances tickets through Pending -> In Progress -> Resolved.

HostelFlow — Technical Specifications

Directory Mapping, Database Schema, and Local Execution

4. Where Everything Lives in the Codebase

- **client/src/pages/student/:** Student views: StudentDashboard.jsx, BrowseRooms.jsx, PaymentPage.jsx, MyComplaints.jsx.
- **client/src/pages/warden/:** Warden views: WardenDashboard.jsx, ApplicationsQueue.jsx, PaymentVerificationQueue.jsx, ComplaintsManagement.jsx, Analytics.jsx.
- **client/src/api/:** Modular Axios client functions matching backend endpoints (auth.api.js, room.api.js, application.api.js, payment.api.js, complaint.api.js).
- **server/src/controllers/:** Business logic: application.controller.js (atomic transaction lock), payment.controller.js, auth.controller.js, complaint.controller.js.
- **server/src/middleware/:** Security layers: auth.middleware.js (JWT parsing), role.middleware.js (Role guards), upload.middleware.js (Multer file storage).
- **server/prisma/dev.db:** SQLite relational database storing Users, Students, Wardens, Rooms, Applications, Payments, and Complaints.
- **start-all.bat:** One-click Windows launcher that runs both frontend and backend and launches the browser.

5. Pre-Seeded Demo Credentials

Warden Account:
Email: warden@hostel.edu | Password: WardenPass123! | Name: Dr. Alok Verma

Student Account:
Email: student@college.edu | Password: StudentPass123! | Name: Rahul Sharma (21BCE1042)

6. Academic & Viva Presentation Highlights

- **Why External Portal Redirect?:** College campuses use centralized ERP fee systems (MMMUT Samarth). Since external apps do not have institutional banking API access, the unique challan redirect model is the industry standard for payment reconciliation.
- **Double-Booking Prevention:** Room capacity checks and seat incrementing are encapsulated inside an atomic Prisma $transaction. Race conditions between concurrent applicants are mathematically impossible.
- **Audit Trail:** Every administrative action logs staff IDs and timestamps (decidedAt, verifiedAt, resolvedAt).

HostelFlow — 100% Operational & Verified